

领域知识个性化生成与
多智能体协同决策系统
技术解读与交接说明

榜题编号 XH-202630

发榜单位：上海云之脑智能科技有限公司

2026 年 8 月

前言

这份文档写给要接手把这套系统做完的同学，不是写给评委看的宣传材料。

系统的骨架已经跑通：五类智能体、显式状态机、审核与辩论双重防线、自适应测评、命题闸门、内容流水线，共 237 个单元测试全部通过，不依赖任何第三方包。但骨架跑通与作品完成之间还隔着一段路，这段路的主要工作量不在写代码，而在填内容——知识库要从二十六条切片扩到三百条以上，题库要从四十七道扩到九十道以上，还要补人工标注。这部分只能由懂领域、手上有真实资料的人来做。

因此本文档的组织方式与一般的技术说明不同。它不只讲系统怎么运转，更着重讲三件事：每一处设计为什么这么做，开发中踩过哪些坑，以及哪些数字不能拿去作为效果证明。第三件事尤其要紧。

必须在开篇就写明的一条是：本文档中的全部评测数字，目前都不能作为效果证明。原因在 8.1 节详述——知识库当前二十六条切片中仅五条经过来源核实，核实率百分之十九。整套幻觉检测的逻辑是「断言必须被知识库支撑」，那么知识库本身若是错的，审核闸只会把错误认证为正确。「幻觉率百分之零」的真实含义是「生成内容与知识库一致」，而不是「内容正确」。这两句话差得很远，而对外材料极易把前者说成后者。评测脚本会在核实率不足时主动打印这段警告。

第八章第三节列出了当前已知的全部短板。答辩时主动讲短板，比被评委追问出来强得多；含糊过去，一旦被抽样问到就很难看。

书中若干设计取舍源于开发过程中的实际失败，附录汇编了十六条。这些记录并非自曝其短，而是这套系统最有说服力的部分——它们说明系统确实被跑通过、被压测过，而不只是搭了个架子。

[编写者]
2026 年 8 月

目 录

前 言	iii
1 绪论	1
1.1 榜题要求与本系统的定位	1
1.2 总体架构	1
1.3 三条不能破的原则	2
1.3.1 大模型只管生成内容，不管控制流	2
1.3.2 没有引用的断言等于没说	2
1.3.3 检索按知识点硬过滤	2
1.4 运行方式	2
参考文献	3
2 学情建模	5
2.1 自述解析	5
2.2 贝叶斯知识追踪	5
2.3 证据强度：点估计不足以下结论	6
2.3.1 判定门槛	7
2.4 自适应选题	7
2.5 动态追问	7
2.6 能力图谱	8
2.7 前置推断	8
参考文献	8
3 领域知识组织	11
3.1 检索与断言	11
3.2 溯源可信度：压倒其余全部指标的一件事	11
3.3 核实状态的五档取值	12
3.4 能不能让模型自动核实	12
参考文献	13
4 多智能体协同	15
4.1 为什么不做“自由辩论”	15
4.2 判定“一致”的三个条件	15
4.3 辩论机制的实测效果	15
参考文献	16

5 内容审核与幻觉防控	17
5.1 四层判据	17
5.2 红队测试：拦不住的是哪一类	17
5.3 三种资源形态	18
参考文献	18
6 命题与题目质量	19
6.1 让模型出题的风险	19
6.2 五关审核	19
6.3 否定感知：最隐蔽的一个坑	20
6.4 题目质量：题不好，尺子就不准	20
6.4.1 上线第一天就抓到的两个问题	20
6.4.2 结构检查与实测标定	21
参考文献	22
7 内容流水线	23
7.1 原来的问题：太封闭	23
7.2 三道关口，各查各的	23
7.3 交叉校验的一处逻辑倒置	24
7.4 剔除靠隔离，不靠删除	24
7.5 摄入产出的状态是 sourced	24
参考文献	24
8 评测、短板与后续工作	25
8.1 哪些数字能说，哪些不能说	25
8.2 只有人能供给的两样东西	25
8.2.1 知识库导入器	26
8.2.2 人工评分工具	26
8.3 已知短板清单	26
8.4 接入大模型	26
8.5 排期与分工	28
参考文献	28
A 踩坑记录汇编	29

第1章

绪论

1.1 榜题要求与本系统的定位

本系统面向挑战杯“揭榜挂帅”擂台赛榜题 XH-202630[?]. 榜题要求构建一套面向垂直领域技能培训的领域知识个性化生成与多智能体协同决策系统，核心指标有三项：专业知识谬误率低于百分之五、学习者画像与资源难度适配准确率不低于百分之八十五、核心知识点覆盖率不低于百分之九十。

榜题在评分标准中明确点名了两项技术要求。其一是引入“辩论与交叉验证”机制以消除大模型在专业领域的幻觉；其二是探索大模型驱动下的动态追问与启发式交互导学，打破静态资源的单向输入局限。这两项在本系统中分别对应第四章的双专家交叉验证与第二章的自适应测评。

本系统选取工业机器人现场操作与调试作为示范领域。选择这一领域有其技术上的理由：它拥有大量封闭事实集，例如报警代码表、指令参数表、公差要求与保养周期。封闭事实集的特点是对错一查便知，因而幻觉率的真值容易建立；若选取开放性题材，幻觉率将无从测量。

1.2 总体架构

系统由五类智能体与一个编排层构成，各自职责见表 1.1。表中最后一列尤其值得注意：承担判断职责的环节全部不使用大模型。

表 1.1 各智能体的职责划分

智能体	职责	是否用大模型
学情访谈	自述文本转结构化背景	用，有规则兜底
学情诊断	作答记录转掌握概率，盲区排序	不用
领域专家甲	窄检索起草断言	用
领域专家乙	宽检索起草断言	用
交叉验证裁判	对齐两份断言，仲裁冲突	不用
审核裁判	逐条断言核验	规则为主，模型为辅
命题	现场出变式题，过五关审核	用
决策调度	按正确率决定降维、巩固或进阶	不用

全流程的状态流转如下：

```
INIT -> DIAGNOSE -> PLAN -> [ GENERATE -> DEBATE -> AUDIT  
-> (REVISE -> AUDIT)* -> ASSEMBLE ]* -> READY
```

READY -> FEEDBACK -> DECIDE -> GENERATE ...

其中 GENERATE 一步由两位领域专家并行执行，专家甲走窄检索、专家乙走宽检索；DEBATE 对二者产出的断言做交叉验证与仲裁；方括号表示该段对学习路径上的每个知识点重复一次；星号表示该段可能重复零到两轮。反馈回路从 READY 出发，经决策调度后重新进入生成，构成动态迭代。

编排层采用手写状态机，未采用 LangGraph 或 AutoGen 一类框架，理由有三。其一，每条状态转移边都能直接写成单元测试，而榜题明确要求测试多智能体协同调度逻辑。其二，出现问题时调用栈很浅，学生自己即可调试，不必先读框架源码。其三，状态与事件由本系统自行定义，前端可视化与评测中间数据可以复用同一份结构。待真正需要并行调度或人在回路时再行迁移，接口已经预留。

1.3 三条不能破的原则

以下三条原则是整个方案可信度的来源。后续增加功能时守住它们，比多加两个智能体有用得多。

1.3.1 大模型只管生成内容，不管控制流

掌握度是算出来的，教学决策是规则判的。若将其交给模型，同一份前测运行两遍结果不一致，所谓“个性化适配”与“动态迭代”就变成了看运气，评委随手试两次便会露馅。

落到代码上：贝叶斯知识追踪的掌握度估计、自适应选题、追问触发条件、降维与进阶的阈值判定，全部是算术，没有一行调用模型。模型只负责把算好的数字翻译成自然语言，以及产出内容文本。

1.3.2 没有引用的断言等于没说

生成智能体的最小输出单元是断言，每条必须携带知识库切片编号。审核不通过就打回重写，最多两轮，仍不通过则直接丢弃。

幻觉率能够压住，是因为架构上根本出不去，而不是生成完再去统计有多少条是错的。这是技术创新性评分项的核心论据。

1.3.3 检索按知识点硬过滤

生成某个知识点的资源时，模型只看得到该知识点的切片，跨知识点的内容串味在检索阶段即被切断。这是压制幻觉的第一道闸门，实现成本几乎为零。

1.4 运行方式

系统不需要安装任何第三方包，不需要联网，也不需要模型接口密钥。解压后进入项目目录，执行下列命令即可验证全部功能。

首先运行一次完整闭环，观察多智能体调度的时间线。两位学习者基础不同，生成的学习路径也完全不同：


```
python3 cli.py P-A  
python3 cli.py P-C
```

其次运行全部单元测试，当前共 237 个用例：

```
python3 -m unittest discover -s tests -v
```

三类评测分别为三项指标的批量评测、按幻觉类型统计检出率的红队测试、以及题库质量报告：

```
python3 -m evalkit.run_eval --n 50  
python3 -m evalkit.redteam  
python3 -m evalkit.itemreport
```

消融对照是评测中最有说服力的部分，分别关闭审核环节与辩论环节，观察各自对幻觉率的贡献：

```
AGENTEDU_INJECT=0.3 python3 -m evalkit.run_eval --n 50 --no-audit  
AGENTEDU_INJECT=0.3 python3 -m evalkit.run_eval --n 50 --no-debate
```

最后启动演示服务，浏览器访问 <http://127.0.0.1:8000>：

```
python3 server.py
```

最重要的一个文件：web/showcase.html

双击即可打开，不需要后端、不需要网络、不需要安装任何东西。评委可以现场输入自己的背景、亲手答题，系统当场算出学情并生成资源。

这是专为答辩现场准备的。演示服务在别人机器上起不来是很常见的事，端口占用、防火墙策略、Python 版本差异都可能导致失败，单文件版把这个风险彻底消掉。后端在线时它会自动切换为实时模式，页面顶部的指示灯显示当前数据源。

参考文献

第2章

学情建模

2.1 自述解析

学习者输入一段自我描述，系统从中抽取学历、阶段、专业方向与实操学时。配置了模型接口时走模型抽取再用规则校验，未配置时纯规则运行，离线一样可用。

需要强调的是，抽取结果只用来给初始掌握概率定先验，不直接决定结论。先验会随着作答证据的累积被冲淡，因此即使抽错也不会一错到底。这是有意的设计：自述是主观的，学习者容易高估或低估自己，不能让它拍板。

踩坑：「二年级」被算成两年工龄

自述“高职电气自动化二年级”当中的“二年”曾被正则表达式当作时长，折合三千二百小时实操，先验直接顶到取值上界，测评开局就问偏了难度。学制与工龄必须分开，现在的做法是在时长模式后加负向前瞻，排除“级”“段”“检”等后缀。

2.2 贝叶斯知识追踪

掌握度估计采用贝叶斯知识追踪[1]，而非简单的答对率。答对率有两处硬伤。其一是蒙对算掌握：四选一题目的蒙对概率为四分之一，答对率把这部分当成了真会。其二是失误算不会：真正掌握的人手滑点错，答对率会直接把他判成盲区。

贝叶斯知识追踪用四个参数把这两件事显式建模，各参数含义见表 2.1。

表 2.1 贝叶斯知识追踪的四个参数

参数	含义	说明
p_{L0}	初始掌握概率	由学历与实操学时给出先验
p_T	学习跃迁概率	前测阶段无教学干预，影响很小
p_S	失误率	已掌握却答错
p_G	蒙对率	四选一题目的理论下界即为 0.25

设 $P(L)$ 为当前掌握概率，一次作答后的后验与学习跃迁按下式更新：

$$P(L \mid \text{答对}) = \frac{P(L)(1 - p_S)}{P(L)(1 - p_S) + (1 - P(L))p_G} \tag{2.1}$$

$$P(L \mid \text{答错}) = \frac{P(L)p_S}{P(L)p_S + (1 - P(L))(1 - p_G)} \tag{2.2}$$

$$P(L') = P(L \mid \text{obs}) + (1 - P(L \mid \text{obs}))p_T \tag{2.3}$$

必须卡住的一条： $p_S + p_G < 1$

这两个参数之和大于等于一时模型退化，答对与答错所给出的信息方向会翻转，出现“答错反而涨掌握度”的荒唐结果，文献中称为模型退化。代码中的参数校验函数会直接抛出异常拦住此类配置。用真实作答数据重新拟合参数时，这一条必须先卡。

2.3 证据强度：点估计不足以下结论

前一节给出的是掌握概率的点估计。只看点估计会得出荒唐的结论，表 2.2 列出了几种典型作答。

表 2.2 点估计与蒙对概率的对照（四选一题）

作答	点估计	纯蒙达到的概率	80% 区间下界
2/2	0.896	6.2%	0.33
3/3	0.973	1.6%	0.48
3/4	0.856	5.1%	0.26
4/4	0.994	0.4%	0.59

原实现把 2/2 判为“掌握牢固”。但纯靠蒙达到 2/2 的概率有 6.2%，二十个人里就有一个能蒙出来；3/4 判“掌握牢固”，蒙对概率 5.1%，同样站不住。少量作答下的点估计本身就是高方差的，拿它当结论，等于把“目前的最佳猜测”说成“已经确认”。

因此每个知识点除点估计外，必须同时给出两个数。蒙对概率即完全不会的人靠蒙达到该成绩或更好的概率，按二项分布右尾计算，这是最直观的一个数，直接回答“我瞎蒙也能考成这样吗”。可信区间即掌握概率的区间估计，其下界才是能对外声称的下限。

区间的算法是：作答正确率 θ 的后验取 $\text{Beta}(1 + k, 1 + n - k)$ ，再按观测模型反解回掌握概率：

$$\theta = p_L(1 - p_S) + (1 - p_L)p_G \implies p_L = \frac{\theta - p_G}{1 - p_S - p_G} \tag{2.4}$$

这一步反解正是扣掉蒙对成分。分母必须为正，也就是 $p_S + p_G < 1$ ，否则反解方向会翻转。

区间水平取 0.80 而非常用的 0.95：本项目的作答数极少，0.95 区间在 $n = 2$ 时几乎覆盖全域，虽然正确但没有任何决策价值。

2.3.1 判定门槛

声称“已确认掌握”要同时满足三条：点估计过线、区间下界也过线、且蒙对概率不高于 5%。三条里最容易被忽略的是第二条——点估计 0.896 而下界只有 0.33 时，说“已掌握”是没有依据的。不满足则一律降级为“疑似掌握”并标注证据不足。

判盲区不必卡蒙对概率——蒙对只会把成绩往上抬，成绩低说明连蒙都没蒙上，结论方向是安全的；但一题作答仍不足以定论。

一个必须正视的结论：要确认掌握需要八题全对

按上述门槛推算，80% 区间下界要过 0.80 这条线，需要连续八题全对。

这意味着现在每知识点三道题的题库，结构上就不可能“确认”任何一个知识点。系统当前对所有知识点给出的都是“疑似掌握”或“证据不足”。这不是模型效果问题，是题量问题。把题库扩到每知识点五到六道之所以列为最高优先级之一，原因即在于此。

在此之前，界面与报告中一律使用“疑似掌握”，不得写“已掌握”。

2.4 自适应选题

系统不会把全部题目一次性发给学习者，而是每次只发一道，根据已有作答实时挑选下一道。设当前掌握概率为 p_L ，则预测答对概率为

$$P(\text{答对}) = p_L(1 - p_S) + (1 - p_L)p_G \tag{2.5}$$

该值越接近 0.5，题目的信息量越大——有把握答对或答错的题目，问了也不会带来新信息。因此选题时取 $|P - 0.5|$ 最小者，再叠加一个“该知识点问得越少越优先”的权重。

2.5 动态追问

动态追问直接对应榜题创新性条款中“大模型驱动下的动态追问”这一要求。当一次作答与已有证据明显冲突时，系统就地追加同一知识点的一道题目，两类触发情形见表 2.3。

表 2.3 动态追问的触发条件

情形	歧义	动作
估计已掌握（不低于 0.65）却答错	是失误还是真不会	追问一次
估计为盲区（不高于 0.35）却答对	是真会还是蒙对	追问一次

贝叶斯知识追踪的失误率与蒙对率恰好把这两件事显式建模了，因而歧义可以量化。追问不是随便加一道题，而是为消解特定歧义而加。

踩坑：追问在第一题就触发

最初没有限制触发前提，结果先验偏低时（零基础学员约 0.20），任何一道答对都满足“估计为盲区却答对”，第一题就触发追问。但那时手上一条证据都没有，所谓冲突是与一个通用先验冲突，消解不了任何歧义。实测十六道题中有七道是追问，知识点覆盖率被吃掉近一半。修法是加一道门槛：该知识点必须已有作答证据才允许追问。

2.6 能力图谱

逐条列出十五个知识点的掌握概率，信息是全的，但没人读得进去——一屏数字，看不出这个人强在哪、弱在哪。因此把知识点聚合成六个能力维度，以雷达图呈现。

雷达图有个众所周知的毛病：它很好看，因而特别容易骗人。本系统用三条设计压住这一点。

轴不能太碎。不直接用知识点标签做轴——现有十三个标签里有八个只覆盖一个知识点，画出来大部分轴由单点决定，一道题的对错就能让整条轴塌掉，看着像能力图谱，其实是噪声图。归并到六维、每维至少三个知识点才有平均的意义。

必须画两层。外层是点估计，内层是区间下界，两层之间的间隙就是“还不确定部分”。只画点估计会让四道题答对三道看起来和真正掌握一模一样，而这两件事的证据强度差着数量级。

未测的维度不能画成零。零和“没测过”在雷达图上长得一样，但含义相反：前者是“确定不会”，后者是“不知道”。未测部分单独用虚线标出，并在图例中写明。

六个维度分别为基础认知、现场操作、安全规范、精度标定、编程集成与运维调试，合计覆盖全部十五个知识点。维度得分取该维度下各知识的平均，未测的知识点不计入平均，而是单独记进覆盖率——把未测当零会让没考的维度显示成“完全不会”，这是雷达图最常见的误导方式。

2.7 前置推断

十五个知识点若要各自测到可信度 0.75，需要每点三题、合计四十五题，没有学习者愿意作答。因此系统增加了一条规则：前置知识点已确认为盲区的，不再实测，直接推断为盲区。连坐标系都分不清的学习者不可能掌握工具坐标系四点标定，花一道题去确认这件事是浪费。

推断得出的盲区在界面上用虚线框标出，不冒充实测结果，报告中亦应分开统计。

参考文献

- [1] CORBETT A T, ANDERSON J R. Knowledge tracing: Modeling the acquisition of procedural knowledge[J]. User Modeling and User-Adapted

Interaction, 1994, 4(4) : 253 – 278.

第3章

领域知识组织

3.1 检索与断言

检索采用纯标准库实现的 BM25，中文按字符二元组切分。刻意不使用分词与向量检索库，理由是要保证评委在自己的机器上克隆下来即可运行，少一个依赖就少一个现场翻车的可能。后续升级只需替换检索接口的实现，调用方不变。

生成智能体输出的最小单元是断言，每条必须可独立判断真伪，且携带知识库切片编号。

3.2 溯源可信度：压倒其余全部指标的一件事

知识库本身没核实，评测数就不是效果数

整套幻觉检测的逻辑是“断言必须被知识库支撑”。那么反过来，知识库本身如果是错的，审核闸只会把错误认证为正确。跑分中的“幻觉率百分之零”，真实含义是“生成内容与知识库一致”，不是“内容正确”。这两句话差得很远，而对外材料极易把前者说成后者。

骨架版交付时，二十六条切片中有二十一条的出处是编造的占位文本，形如“《工业机器人操作与运维》第4章”，看上去毫无破绽。这比明显的胡编更危险，因为它会让读者、评委、甚至团队自己误以为内容有据可查。

现已联网核实，结果见表 3.1。

表 3.1 知识库核实结果

条目	结论	说明
四个报警代码	内容正确	SRVO-001、002、005、062 分别对应操作面板急停、示教器急停、机器人超程、脉冲编码器备用电池电压不足，与多份厂商资料一致
示教限速	内容正确	示教时运动速度应低于 250 mm/s[?]
围栏高度与安全距离	出处错配	原稿把数值挂在一条未经核实的国标条款名下。内容或许合理，但挂到具体标准上属于错配
其余二十一条	未核实	含标定公差、润滑周期、子程序层数上限等

核实过程中出现了一个值得记住的现象：某维修服务商网站把 SRVO-001 说成“伺服放大器过载”，与多份官方手册摘录完全矛盾。单一网络来源不足以定案，这不是理论上的顾虑。

3.3 核实状态的五档取值

每条切片现在都带核实状态字段，取值分五档，见表 3.2。

表 3.2 核实状态取值与写入权限

状态	含义	谁能写
unverified	未核实（默认）	机器
sourced	来自真实文档，可回溯到文件与位置	机器
machine_checked	找到外部证据且多来源一致	机器
disputed	证据不足、来源单一或多来源矛盾	机器
refuted	找到明确矛盾的外部证据	机器
verified	人工确认无误	只有人

最后一行是整个设计的支点。一旦机器能写 `verified`，整条溯源链就退化为“模型说它查过了”，与不核实无异；写回函数中有断言直接拦住此类写入。

`sourced` 与 `verified` 的区别也需要讲清楚。从真实文档切出来的内容是“有出处的”，不等于“经过核实的”——出处真实只说明这句话确实印在那本书上，不说明那本书是对的。但 `sourced` 本身已是巨大进步：它的出处是文件名加页码加文件指纹，任何人都能翻回原文核对，而骨架版那二十一条是查无此书。

3.4 能不能让模型自动核实

能，但有一条铁律，破了就前功尽弃：模型的判断本身永远不能成为核实依据，核实依据只能是外部证据。

直觉做法是把切片丢给大模型问“这条对不对”，模型说对就标成已核实。这么做比不核实还糟。知识库本来就是大模型写出来的，再让大模型判它对不对，等于让同一个来源自己给自己背书——模型倾向于认可符合自己先验的说法，而那些说法恰恰是它自己生成的。结果是把“未核实”洗成“已核实”，标签变了、可信度一点没变，错误从此戴上一顶合规的帽子，比裸奔更难被发现。

核实智能体按这条铁律实现。模型在其中只干两件事：把切片改写成检索词；读取检索回来的外部资料判断是否支持该切片。第二件事仍是判断，但判断对象是别人写的资料，而非自己的记忆。检索不到资料就判存疑，绝不允许回落 to 模型记忆。

判定遵循三条规则。多来源：至少两个互相独立的域名一致才算通过，同一域名下的多个页面不构成独立来源。矛盾优先：只要有一个来源给出矛盾结论就判被推翻，哪怕支持方更多；测试中构造过三比一的场景，矛盾方胜出。无证据不放行：检索源没接上时全部判存疑，这是正确行为而非故障。

矛盾优先这一条对应本项目真实踩过的坑：上一节提到的 SRVO-001 争议，当时是人发现的，现在系统能自动把它顶出来并附上证据清单。

这个工具是省人力的，不是替人拍板的

它把证据摆到人面前并排好优先级，最后那一下还得人来点——检索到的网页本身也可能是错的。

参考文献

第4章

多智能体协同

4.1 为什么不做“自由辩论”

榜题在创新性条款中点名要求“交叉验证与辩论”机制。本方案没有让两个模型来回争辩三轮——那样既消耗大量令牌，又无法测量，对准确率也没有帮助。这里实现的是断言粒度的对抗与仲裁。

第一步是两位专家独立起草。关键在于“独立”：专家甲按知识点硬过滤取前若干条切片，视角窄而准；专家乙全库检索后按知识点重排，视角宽而全。视角不同，犯的错才不同。两个智能体如果看到同一份上下文，等于同一个模型问了两遍，没有任何信息增益。

第二步是对齐。按相似度将两份断言配对，分为双方都提出、仅一方提出、双方冲突三类。

第三步是仲裁。裁判不投票，也不看谁表述得更漂亮，只回到知识库比对证据。证据分差不足时两条都弃用。

4.2 判定“一致”的三个条件

整个机制里最关键的一行

判定一致必须同时满足三条：文本高度相似、出处相同、数值集合相同。

第三条是被单元测试逼出来的。原先的实现只看前两条，结果专家甲说“限速 250 mm/s”、专家乙说“限速 600 mm/s”，文本相似度高达 0.95、引用出处又完全相同，被判成“双方印证”直接放行。数值篡改恰恰是文本最相似的一类冲突，只看相似度等于把最危险的情形当成了最可信的情形，方向完全反了。

对应地，仲裁分两层：数值集合不同时，以“谁的数字在知识库”定胜负，黑白分明；其余情况才比较整体证据分。

4.3 辩论机制的实测效果

关闭审核环节、只观察辩论环节的分层统计，在同一批 9168 条断言上的结果见表 4.1。

两者相差约三十五倍。这说明“两位专家是否独立提出同一条”本身就是一个强幻觉信号，交叉验证不是走过场：一条被编造出来的断言，另一位采用不同检索视角的专家几乎不会编出同一句。

表 4.1 按共识等级分层的无依据率

共识等级	断言数	无依据条数	无依据率
双方独立印证	5001	27	0.54%
仅单方提出	4167	786	18.86%

必须写进报告的限制

上面这组数是离线桩跑出来的。桩按种子给两位专家注入不同的假事实，“独立编造不会撞车”在桩里部分是构造出来的。真实模型下这个倍数会变，接上真模型后必须重测，不要把三十五倍当作结论。

参考文献

第 5 章

内容审核与幻觉防控

5.1 四层判据

大模型在垂直领域的翻车方式是有限的几种，审核判据逐一对应，见表 5.1。

表 5.1 审核裁判的四层判据

失败模式	判据	配置项
完全编造，不给出 处	无切片编号直接判无依据	无
引用不存在的切片	切片编号查不到	无
数值被篡改	断言中的数字必须在切片中出现过	NUMERIC_STRICT
张冠李戴	术语归属检查与全库最佳匹配比对	TERM_STRICT

踩坑：中文数字从数值检查底下溜过去

原先的数值核验只识别阿拉伯数字。“减速机更换周期为运行两万小时或五年”改自“一万小时或三年”，整句没有一个阿拉伯数字，直接绕过了检查。中文技术文档里汉字数字极其常见，这个漏洞不补，数值篡改类幻觉能漏掉一半。现在检索层做了中文数字归一化，汉字与阿拉伯数字统一折算后比对。

5.2 红队测试：拦不住的是哪一类

只报告总体幻觉率，评委无法判断团队是真解决了问题还是数据本身简单。红队测试按幻觉类型分别统计检出率，结果见表 5.2。

表 5.2 按幻觉类型的检出率

类别	检出率	说明
H1 凭空捏造	100%	无出处，结构性堵死
H2 引用悬空	100%	切片不存在，结构性堵死
H3 数值篡改	100%	数值核验，含中文数字归一化
H4 张冠李戴	100%	术语归属与全库最佳匹配比对
H5 过度泛化	67%	纯规则拦不住，需要蕴含判断
H6 似真外推	67%	同上，且最危险

对照十条真断言，误伤零条，总检出率为 90.5%。

H5 与 H6 是当前防线的真实短板，报告中要如实写明。写清楚短板反而加分，因为这说明团队真的测过；含糊过去，一旦被评委抽样问到就很难看。

5.3 三种资源形态

三种资源形态共享同一个已过审的断言池，套用三套模板产出：要点讲义逐条附带引用编号与出处；实操指南包含前置条件、操作步骤、常见错误与安全提示；分阶测试题按难度排序，每题挂接溯源信息。

如此处理之后，榜题要求的“至少三种形态”几乎不增加额外的幻觉风险，也不必审核三遍。

参考文献

第6章

命题与题目质量

6.1 让模型出题的风险

题库只有四十七道题，每个知识点三道，难度档位有限。学习者水平落在两道题之间时，只能取一道偏难或偏易的凑合，而太难必错、太易必对，那一问的信息量就白费了。因此引入模型现场命制变式题目，把题库当作种子而非天花板。

先说风险，这条比什么都重要

模型出题等于模型定义“正确答案”，这是本项目中后果最重的一类错误。

讲义里写错一句，学习者可能看得出来，审核也拦得住。但答案标错了，系统会拿一把错的尺子去量人：学习者答对却被判错，掌握度往下走，系统随后给他推送更简单的内容；学习者答错却被判对，盲区被漏掉，该补的没补。

错误不会停留在一句话上，而是灌进学情模型，决定后面所有资源。学习者还无法申诉，因为他并不知道标准答案是编出来的。

所以规矩是：生成题必须过闸，一道都不例外；过不了就丢弃，回退固定题库。宁可用一道糙题，不能用一道错题。

6.2 五关审核

命题审核的五道关卡见表 6.1。前四关问的是“这道题对不对”，第五关问的是“这道题好不好用”，后者详见 6.4 节。

表 6.1 命题的五关审核

关	查什么	为什么
1	正确答案被所引切片支撑	沿用审核裁判那套判据，不给命题开小灶
2	每个干扰项都不成立	一题两解比答案错了还糟，且没有环节会发现
3	题干不含切片以外的数值	只查题干，不查选项
4	选项不重复、无长度线索	最长的恰好是答案，学习者不会也能蒙对
5	结构质量	见 6.4 节

踩坑：数值型选项不能用文本覆盖率判

“0.5”这样的纯数字选项，按字符二元组切分后只剩“0”和“5”，只要切片正文任意位置出现过这两个字符，覆盖率就是满分，判据完全失效。数值型选项的正确判据是“这个数在不在知识库里”，黑白分明，不该绕道文本相似度。因此审核对数值型与文字型选项分开处理。

第三关只查题干不查选项，这一点同样是踩出来的。干扰项按定义就该带上知识库里没有的数值，拿这一条去卡选项等于禁止出错误选项，第一版就是这么把所有题目全部毙掉的。同理，长度线索规则跳过纯数值选项：一万比二长，但没有哪个学习者会因此认为它是答案。

6.3 否定感知：最隐蔽的一个坑

在安全与规程类内容里，最好的干扰项恰恰是手册明令禁止的做法。知识库中记载“切勿在未按住解除按钮时强行驱动”，那么“直接强行反向驱动”就是一个再标准不过的干扰项。可它与该切片的字符二元组覆盖率高达 0.60，被判成“可能同样成立”而毙掉。

问题在于二元组重合看不见否定，“强行驱动”与“切勿强行驱动”在词面上几乎一样。照这样卡下去，模型只能去造一眼就假的干扰项，题目的区分度随之归零。

修法是把切片按分句切开，比较选项与“禁止句”和与普通正文的重合度，更像禁止句就放行。这里用比较而不是绝对阈值，因为干扰项通常只借用禁止句的一部分措辞，实测那一句只有 0.40，阈值卡在 0.5 会漏，卡在 0.3 又会误判，比较式天然免疫这个标定问题。

两条反向保护同样重要。其一，选项自身带有禁止词说明它在复述规则、本身成立，不享受豁免。其二，切分粒度必须到分句：“加注时必须打开排脂口，禁止在封闭状态下加注”这句话若只按句号切分，整句都会被当成禁止句，连正确做法都被判成“被禁止”，等于把正确答案塞进了干扰项。

6.4 题目质量：题不好，尺子就不准

命题闸门管的是“题对不对”，题目质量模块管的是另一件事：题好不好用。

两者不能互相替代。一道题可以完全正确、出处清楚、干扰项都不成立，却仍然测不出任何东西：太简单人人都对，干扰项一眼就假，两个干扰项几乎一样。这些不是错题，闸门放它们过去是对的。但拿它们去估计掌握度，蒙对率就不再是四分之一，估出来的掌握概率整体偏高，而且偏多少不知道。

6.4.1 上线第一天就抓到的两个问题

四十七道题里三十八道答案在 B

学习者一路选 B 就能得到八成以上的分数，整套测评的测量效力等于零。这是项目自己手写的题库，写的时候谁也没注意到。现已用确定性置换打散，同时把三份画像里存储的作答下标一并重映射——不同步重映射的话，对错模式会静默改变，得到一批“通过但含义已变”的测试，是最难排查的一类问题。

所有判断题的答案都是“正确”

这一个更糟：位置偏斜至少还有四分之一的蒙对下限，答案全同则是百分之百。根因是题干由已过审的断言直接生成，自然全是正命题。

修法是把审核闸反过来用：扰动断言中的数值或特征术语，再送回审核裁判，只有被判定为与知识库冲突才采纳。也就是说，这条假命题的“假”是知识库认证过的，不是我们自己认为它假。认证不了就不产出反向题。现在真假比为八比四。

6.4.2 结构检查与实测标定

结构检查不需要作答数据，出题当场就能算，检查项见表 6.2。

表 6.2 题目结构瑕疵检查项

代码	查什么	为什么
LENCLUE	正确答案明显更长	应试技巧就能蒙对
CATCHALL	“以上都对”一类兜底项	测的不再是本知识点
ABSOLUTE	干扰项绝对化措辞	“所有”“绝不”会被直接排除
DUPDISTRACT	两个干扰项几乎一样	名义四选一、实际三选一
FARDISTRACT	干扰项离题	一眼能排除，白占一个选项位
SHORTSTEM	题干过短	缺少作答语境

判定干扰项重复的阈值取 0.60 而非直觉上的 0.8，这里有个标定细节：中文短选项用字符二元组的 Jaccard 相似度时，改一个字就会掉两个二元组。“由三个直线轴构成”与“由三个直线轴组成”实际是同一个选项，相似度却只有 0.667；而真正不同的两个干扰项是 0.304。两档之间间隔足够大，0.60 落在中间。

实测标定需要作答数据，包含两项指标。难度用 p 值表示，即答对比例，注意方向与直觉相反—— p 越大题目越简单，报告中要写清楚，避免读反。区分度用点二列相关系数表示，含义是答对这道题的人在整份测评上是否也考得更好。

区分度是发现答案标错的唯一可靠手段

对本项目尤其要紧：生成题的答案是模型断言的，标错了不会有任何环节报错。一旦标错，水平高的人反而更容易选到真正正确的那个选项而被判为错，于是相关系数变成负数。

判读标准为：大于 0.30 区分度良好；0.15 到 0.30 可用；0 到 0.15 几乎不提供信息；小于 0 时应优先怀疑答案标错，而不是认为题目难。模拟验证显示，一道答案标错的题目区分度为 -0.149，被直接判为不可用。

题库质量报告的当前结果为：共四十七道，平均质量分 85.7，判为不可用零道，答案位置分布均匀，待改十九道，主要问题是干扰项离题与长度线索。这部分是人的活——改写干扰项使其长度接近、具备干扰力，属于命题技术，模型代劳不了。

参考文献

第7章

内容流水线

7.1 原来的问题：太封闭

在本章所述工具出现之前，往知识库中添加内容必须手写特定格式的 Mark-down，而且知识点字段必须填一个已经存在的编号。这是个死循环：要加内容得先有知识点表，而知识点表写死了十五个。拿到一本真实手册想录进去，第一步就卡住——不知道该往哪个知识点挂，也不知道能不能新建。

现在反过来做：先喂原始资料，让知识点从语料里长出来。

```
python3 -m tools.ingest raw/
python3 -m tools.ingest raw/ --apply
python3 -m tools.kb_audit
python3 -m tools.kb_audit --quarantine
python3 -m evalkit.factcheck_run
```

依次为：切片归类质检并暂存、确认后写入知识库、交叉校验查库内矛盾、把冲突条目移进隔离区、对外核实并产出复核队列。支持 .txt .md .pdf .docx .csv .tsv .xlsx 七种格式。PDF 按页读取，位置标签带页码，这是评委最可能要求当场翻阅的定位方式。

7.2 三道关口，各查各的

三个工具的分工见表 7.1。

表 7.1 三道关口的分工

工具	查什么	看不见什么
tools.ingest	摄入质量：乱码、目录页、无句读、重复	内容对不对
tools.kb_audit	切片之间是否自相矛盾	与外部资料是否一致
evalkit.factcheck_run	与外部资料是否一致	库内两条是否打架

第二道必须单独做。单条格式完全合规、出处也真实的两条切片，可以互相打架——同一个报警代码在甲页写“急停”、在乙页写“过载”，两条都能过入库门禁，外部核实也可能各自找到支持的来源。只有把它们放在一起比才能发现。

这类矛盾的危害是隐蔽的：检索时两条都会被召回，生成的断言无论采信哪一条都能通过审核（因为确实“有依据”），于是幻觉率照样是零，而输出的内容有一半是错的。

7.3 交叉校验的一处逻辑倒置

踩坑：用相似度当冲突的前提，方向反了

交叉校验第一版写的是“相似度够高才比对”，结果漏掉了最严重的一类矛盾：两条切片都在讲 SRVO-005，一条说是机器人超程，另一条说是伺服放大器过载，相似度却只有 0.175——恰恰因为它们说的完全不同。

正确的判断是：讨论对象相同就该比，比的是它们给出的结论是否一致。相似度只用来区分冲突的类型——高相似度加数值不同是“数值被改”，低相似度是“含义不同”，后者往往更严重。

7.4 剔除靠隔离，不靠删除

自动剔除听起来干净，但删掉的是别人辛苦收集的资料。判据有误、阈值偏严、误伤，都会导致内容悄悄消失而无人知晓。因此不合格的条目一律移进隔离区并写明原因，只有显式加参数才真正丢弃。

冲突类问题还有一条特殊规则：冲突双方中已核实的留下，未核实的隔离；双方核实状态相同时两条都留下，只报告。

踩坑：隔离把已核实的一方也带走了

第一版没有这条规则，结果注入的错误数据把已人工核实的两条一起带下水——错误数据把正确数据挤出知识库，这是最糟糕的一种失败。双方状态相同时机器无从判断谁对，此时该做的是叫人来裁，而不是随便挑一条丢掉。

7.5 摄入产出的状态是 sourced

摄入产出的状态是 sourced：出处是文件名加位置加文件指纹，可以回溯原文。这与 verified 是两回事，理由已在 3.3 节说明。tools.ingest 永远不写 verified=true，有测试盯着。

踩坑：加个字段就把整个系统打死了

端到端验收时发现，摄入工具给切片加了 origin 字段，而数据结构不认这个字段，于是入库之后整个系统起不来——检索、诊断、生成全部无法初始化。单元测试没有覆盖到，因为它们从不真正入库再读回。

知识库是多个工具共同写入的，字段会随工具演进而增加。读取端必须对未知字段容错，否则任何一个工具加字段都会把整个系统打死。

参考文献

第 8 章

评测、短板与后续工作

8.1 哪些数字能说，哪些不能说

在阅读本章数字之前

下列结果的性质是系统自检，证明的是“实现与设计一致”，而不是“输出内容正确”。原因已在 3.2 节说明：知识库当前核实率仅百分之十九。

以五十组用例、百分之三十注入率运行的评测结果见表 8.1。

表 8.1 三项指标与消融对照

配置	幻觉率	适配规则一致性	覆盖率
完整流程	0.00%	100%	100%
关闭审核	8.87%	100%	100%
关闭辩论	0.00%	100%	100%
榜题目标	小于 5%	不低于 85%	不低于 90%

消融对照是评测中最值钱的一栏。只报告“幻觉率为零”没有说服力，报告“关掉审核是 8.87%，开着是 0%”才是能站得住的因果论证。

“适配”那一项是自证的，别把它当效果指标

系统按预先登记的规则计算资源难度，这一项又用同一条规则去校验，所以它必然接近百分之百。它证明的是“实现与规则相符”，证明不了“规则本身合理”。

因此在所有对外材料中，这一项一律写作“适配规则一致性”，不写“适配准确率”。答辩时若说成“我们的适配准确率达到百分之百”，评委追问一句“你拿什么当真值”就会塌。评测脚本在这一项达到百分之百时会主动打印警告，就是为了防止有人顺手把它抄进汇报材料。

所有阈值集中在配置文件中。修改配置文件要在版本库里留一次独立提交，提交时间必须早于评测批次的时间戳。规则先定后测，而不是测完再凑定义，守住了才叫科学定标。

8.2 只有人能供给的两样东西

三项指标当中适配那一项是自证的，幻觉率也只能证明“与知识库一致”。任何自动化检验都绕不开这层循环，因为判据和被判对象同源。打破循环只有一个办法：引入一个与系统无关的判断源，也就是人。

系统为此提供两件仪器，它们不产生判断，只把样本抽好、把干扰去掉、把一致性算出来。

8.2.1 知识库导入器

输入为 Markdown，一条切片一个二级标题。选择 Markdown 而非表格格式，是因为正文里逗号、引号、换行都很常见，表格转义一旦出错就是静默截断。

校验在入库时做，不在事后做。一条不合规的切片一旦进了库，就会被检索到、被生成引用、被审核认证为“有依据”，再想揪出来得翻整条链路；入口拦住只需要一次拒绝。批内只要有一条被拒，整批不入库——半批入库会让编号与核实率都对不上账。

拒收项包括：伪造出处、知识点不存在、长度越界、编号重复、状态自相矛盾。其中伪造出处最为要紧——一个像真的假出处会让所有人误以为内容有据可查，它没有任何视觉破绽，比明显编造更危险。

踩坑：规范和语料对不上

写这个工具时发现，规范初稿写“切片 150 到 300 字”，而骨架版二十六条切片全部落在 71 到 104 字，没有一条符合我们自己写的规范。

改了阈值而不是改切片。理由是“一条切片只讲一件事”这条更根本——一个工业事实用中文讲清楚通常就是八九十字，硬凑到 150 字只能靠注水，而注水的句子会拉低审核的证据覆盖率，反过来伤害幻觉检测。

8.2.2 人工评分工具

盲评有三条设计。隐藏系统结论：评分表里不出现系统给的难度值、掌握概率、共识等级——看到系统答案再打分，打出来的是对系统的认同度，不是独立判断，有测试专门检查评分表不泄漏这些字段。打散顺序：每位评分者拿到的顺序不同，由固定种子决定，可复现。低一致不取平均：一致性系数低于 0.6 时，工具会明确写出“请两位评分者一起过一遍分歧条目，对齐标准后重标”——对不齐的两组数取平均，只会得到一个看着精确的假数。

机器造仪器，人出判断

这与核实智能体是同一个思路。机器能做的是把样本抽好、把证据摆到人面前、把一致性算准，最后那一下必须由人来点。凡是让机器代替人下最终判断的设计，都会在某个环节退化成“模型说它查过了”。

8.3 已知短板清单

答辩时主动讲短板，比被评委问出来强得多。本节建议原样进入方案文档。

8.4 接入大模型

配置模型接口只需三个环境变量，随后运行体检脚本：

表 8.2 已知短板与补法

短板	现状	影响与补法
知识库未核实	核实率百分之十九	压倒其余全部短板。知识库错则审核闸把错误认证为正确，所有幻觉率数字都只是自检
H5、H6 类幻觉	检出率仅 67%	纯规则看不见语义层面的过度泛化与似真外推，需增加蕴含判断层
知识库规模	仅二十六条切片	实际需要三百条以上，覆盖率指标全靠它
题库规模与质量	四十七道，十九道待改	自适应要发挥作用需每知识点五到六道
适配指标自证	结构性达到 100%	需要一项独立于生成规则的检验，或人工评分
共识分层倍数	离线桩产出	部分由桩的构造决定，真模型下必须重测
知识追踪参数	文献常用初值	需用真实作答数据重新拟合
生成题难度	标签是“请求值”	无作答数据可标定，需积累数据后做项目分析
辩论对齐	字符二元组相似度	认不出改写，应换用句向量或蕴含判断
会话持久化	存于内存	进程重启即丢失，演示够用，不是生产实现

```
export AGENTEDU_API_KEY=sk-xxx
export AGENTEDU_BASE_URL=https://<your-endpoint>/v1
export AGENTEDU_MODEL=<model-id>

python3 -m evalkit.doctor
```

未配置密钥时系统照样运行，自动退回离线桩——接入模型是增强项而非必需项，演示现场断网也不会挂掉。

体检脚本包含六项检查，其中最要紧的是命题实测。之所以需要它，是因为接入模型最常见的三类故障——JSON 模式不被支持、模型名写错、触发限流——在全流程中的表现都是“生成结果为空”，极难定位。

选型的关键不是挑一个最好的，而是按任务分档：命题用最强的模型，因为答案标错会污染学情模型；断言起草与核验用中档，有审核闸兜底；自述解析与综合诊断用最便宜的，虽然调用量最大但要求最低。

比赛演示完全不需要付费。免费档真正的约束不是“能不能用”，而是每分钟请求数与总额度，代码为此做了主动限速、调用次数硬上限、请求缓存三条保护。其中调用次数上限尤其重要：额度悄悄耗尽后系统会一路回退规则版，“能跑但已降级”是最坏的一种故障，因为使用者并不知道自己在演示哪个版本。

踩坑：额度用完把整条链路炸掉

命题环节原先没有捕获模型异常，调用失败的异常一路冒到编排层，整个演示白屏。而这个场景下正确的行为是降级：命题本来就是增强项，没有它系统照样能用固定题库跑完。演示现场额度用完还能继续演，和当场白屏，是两回事。

8.5 排期与分工

按九月五日提交倒推的排期见表 8.3。

表 8.3 四周排期

周次	任务
第 1 周	验通演示服务；用摄入工具录入真实资料；扩充知识点表；改完待办的十九道题
第 2 周	接入真模型并运行体检；重跑全部评测；补前端资源正文渲染
第 3 周	人工标注与一致性计算；三张图表打磨；单元测试补到三百个以上
第 4 周	撰写测评报告与方案文档；制作十分钟视频；打包提交

知识库建设是纯体力活，但不能交给不懂领域的人。建议由一人负责确定切片边界与知识点归属，此人必须懂领域；两人负责录入；确定边界的那位再逐条复核知识点归属——归属错了，检索的硬过滤就失效了。第一周就要启动，拖到第三周一定来不及。

演示视频须在十分钟以内，制作时间要留足三天而不是三小时，写好脚本再录，即兴录制一定会超时。

最后是两条合规提醒。项目中的三份学习者画像全部是合成数据，提交材料中应明确写出，零成本即可对应数据合规条款。竞赛总则要求作品拥有完整独立知识产权，而知识库是最容易出问题的地方——教材与手册的原文大段照抄存在版权风险，建议以自行改写的技术要点为主，来源字段标明依据的资料。这件事要早处理，不要等到打包时才想起来。

参考文献

第 A 章

踩坑记录汇编

本附录汇总开发过程中真实发生的问题。这部分写进方案文档很有说服力——它们说明团队真的把系统跑通过、压测过，而不只是搭了个架子。

表 A.1 踩坑记录汇编

问题	表现	根因与修法
辩论把数值冲突判成一致	两条断言相似度 0.95、出处相同，被判“双方印证”放行	数值篡改是文本最相似的一类冲突。判定一致须同时满足文本相似、出处相同、数值集合相同
中文数字绕过数值校验	“两万小时或五年”改自“一万小时或三年”，全句无阿拉伯数字	增加中文数字归一化，不补的话该类幻觉漏掉一半
跨短语二元组误伤真断言	跨短语边界拼出的组合当成特征术语	要求两个字的单字文档频率都低于全库比例，高频虚字自然被排除
适配率假性偏低	演示只取前四个盲区时掉到 50%，批量评测却是 92.84%	判定漏了知识点难度天花板。小样本可视化能暴露大样本平均值藏起来的规格问题
追问第一题就触发	十六题里七题是追问，覆盖率被吃掉近一半	先验低时任何答对都满足条件。加门槛：该知识点必须已有作答证据
学制被当成工龄	先验顶到上界，测评开局问偏	学制与工龄分开，时长模式加负向前瞻
答案位置严重偏斜	四十七道里三十八道在 B，一路选 B 得八成分	确定性置换打散，同时重映射画像中的作答下标
判断题全是正命题	一路点“正确”就是满分	把审核闸反过来用，扰动后必须被判与知识库冲突才采纳
干扰项审核看不见否定	手册禁止的做法覆盖率 0.60 被毙，而它正是标准干扰项	按分句切开，比较选项更像禁止句还是更像正文
纯数字选项判据失效	“0.5”切分后只剩两个字符合，覆盖率恒为满分	数值型与文字型选项分开验证
交叉校验逻辑倒置	用相似度当冲突前提，漏掉含义完全不同的两条	越严重的矛盾两条切片越不像。改为对象相同就比结论
隔离带走已核实的一方	注入的错误数据把已核实条目一起挤出知识库	冲突双方中已核实的留下；双方状态相同时都留下，叫人来裁

续表 A.1

问题	表现	根因与修法
加字段打死整个系统	摄入加了新字段，入库后检索与生成全部无法初始化	读取端对未知字段容错。知识库由多个工具共写，字段会随工具演进
额度用完炸链路	调用异常冒到编排层，演示白屏	命题捕获异常返回空值并回退题库
成本显示为零	四位小数把单次调用舍进误差	改为六位小数
报警代码张冠李戴	句式全对、引用格式也对，仅含义安错	补术语归属与全库最佳匹配两条规则，这类错误人工审核也容易漏

本文档随代码一同交付。代码中的注释记录了更详细的设计理由，遇到不确定的地方优先看注释——每一处非显然的决定都写明了为什么。